

RAPPORT SAE S6.01

Application Web de Chatbot

Conseils et recettes de cuisine

**Ryan Maria Paul | Ivan Veljovic Bulatovic | Mathew Prades | Inès
Marcisz**

Année universitaire 2025-2026

Table des matières

Table des matières	2
I. Introduction	3
II. Analyse des besoins et conception	3
A) Besoins fonctionnels	3
B) Besoins non fonctionnels	4
III. Méthode de travail	5
IV. Architecture et choix techniques	6
A) Frontend Angular	6
B) Backend Flask	6
C) Couche données	7
A) Configuration des modèles	7
B) Système RAG	8
VI. Conteneurisation et déploiement	10
A) Conteneurisation	10
a) Stratégie de conteneurisation	10
b) DockerFile	10
c) Fichier .dockerignore	11
B) Déploiement de l'application	11
VIII. Défis et difficultés majeures	12
A) Qualité des réponses RAG	12
B) Gestion du CORS	12
VIII. Conclusion	13
IX. Annexes	13

I. Introduction

Ce rapport présente notre travail portant sur le développement d'une application web de **chatbot** spécialisé dans le domaine de la **cuisine**. En effet, l'objectif principal est de permettre aux utilisateurs d'interroger l'**assistant conversationnel** sur des questions précises à partir de recettes de cuisine fournies sous forme de fichiers PDF.

L'application repose sur une **architecture RAG** (Retrieval-Augmented Generation), qui combine la **recherche sémantique** dans une base documentaire et la génération de réponses par un **LLM** (Large Language Model). Cette approche permet ainsi d'obtenir des réponses précises et contextualisées, à partir des documents, tout en limitant les réponses extravagantes que les LLM peuvent générer lorsqu'ils sont utilisés seuls.

Ainsi, le projet s'est déroulé en deux grandes étapes, à savoir le **développement** de l'application web d'une part, et l'**intégration** à cette application du modèle spécialisé, via un chatbot d'autre part. Nous avons finalement créé un LLM qui a été intégré à l'application via un chatbot, développé notamment avec **Angular** et **Flask**.

Ce rapport couvre l'ensemble du déroulé du projet : la **récupération**, l'**entraînement** et l'optimisation du **modèle**, le **développement** de l'application web, et enfin l'**intégration** du modèle à l'application.

II. Analyse des besoins et conception

Suite à une **analyse des besoins** que nous avons réalisée en équipe à partir des spécifications du projet, nous les avons répartis en différentes catégories.

A) Besoins fonctionnels

- **Gestion des utilisateurs** : L'application doit permettre à l'utilisateur de créer un compte avec une adresse email et un mot de passe. Une fois inscrit, il peut s'authentifier pour accéder à l'application, et il peut se déconnecter à tout moment.
- **Interface de chat** : L'utilisateur peut poser des questions et recevoir des réponses générées par le LLM immédiatement. Il peut démarrer à tout moment une nouvelle conversation, indépendamment des autres.
- **Gestion de l'historique** : Chaque conversation est sauvegardée automatiquement et associée au compte de l'utilisateur. Il peut consulter ses conversations passées, reprendre une conversation existante avec l'intégralité des échanges, ou supprimer une conversation qu'il ne souhaite plus conserver.

- **Pipeline RAG** : Le pipeline RAG est déclenché à chaque requête de l'utilisateur. Il repose sur des recettes sous forme de fichiers PDF, extraits et découpés en fragments (chunks), génère les représentations vectorielles (embeddings) de chaque fragment, et les indexe dans FAISS. À chaque question, le système recherche les passages les plus pertinents, construit un prompt enrichi à partir du contexte et de l'historique de la conversation, puis appelle le LLM pour générer la réponse.g:

B) Besoins non fonctionnels

En complément des fonctionnalités, plusieurs **contraintes** non fonctionnelles ont été identifiées et ont influencé nos choix techniques :

Critère	Exigence	Solution retenue
Performance	Réponse perçue rapide malgré le LLM	Streaming SSE token par token
Sécurité	Données utilisateurs protégées	JWT + hachage bcrypt
Disponibilité	Application accessible en ligne	Déploiement via Render
Maintenabilité	Code modulaire et versionné	Architecture en couches + GitHub
Scalabilité	Corpus documentaire extensible	FAISS rechargeable + script d'ingestion
Gratuité	Hébergement sans coût	Render

III. Méthode de travail

Dans l'optique de mener le projet dans des conditions proches du monde professionnel, nous avons appliqué une organisation basée notamment sur **GitHub** pour la gestion du code et **Jira** pour la répartition et l'avancement des tâches. Sur GitHub, nous avons adopté une stratégie de branches structurée :

- une branche **main** pour le code stable,
- une branche **develop** pour intégrer les développements en cours,
- des branches **feature** créées depuis **develop** pour chaque fonctionnalité.

Chaque fonctionnalité était développée sur sa propre branche, puis intégrée via **Pull Request**. Ce fonctionnement nous a permis de travailler efficacement même lorsque nous n'étions pas toujours tous ensemble : les autres membres pouvaient relire le code, le tester, poser des questions sur certains choix techniques et proposer des améliorations avant la fusion. Cela a renforcé la qualité du code et la compréhension collective du projet.

En parallèle, nous avons utilisé Jira pour organiser le travail de manière **agile**. Le tableau **Jira** nous a permis de :

- visualiser les tâches restantes,
- prioriser ce qu'il fallait faire,
- répartir le travail entre les membres,
- suivre l'avancement au fil du temps.

Grâce à cette organisation, nous avons gagné en **visibilité**, en **rigueur** et en **coordination**, ce qui a facilité le développement du projet et le respect des objectifs.

IV. Architecture et choix techniques

À l'issue de l'analyse des besoins, nous avons retenu une architecture en **trois couches** distinctes.

A) Frontend Angular

Nous avons développé l'**interface utilisateur** avec le **framework Angular** et déployé sur Render. Angular est un framework basé sur **TypeScript**. Il est particulièrement utilisé pour sa fonctionnalité d'**application web monopage**, qui permet de faire son site sur une seule page que l'on modifie dynamiquement pour l'adapter au besoin. Ainsi, on ne doit pas recharger tous les éléments pour accéder à une nouvelle section.

Cette fonctionnalité est complétée par son **système de signal** qui nous permet de définir les relations entre les éléments. Ainsi, lorsque l'on modifie un élément, Angular ne doit pas parcourir toute la page pour vérifier quels éléments sont impactés par cette modification, mais a juste besoin de regarder les éléments qui sont liés à celui qui vient d'être modifié.

Pour le développement de notre frontend, nous avons décidé de faire l'interface graphique en trois **modules** principaux : le module d'**authentification** (connexion et inscription), le module de **chat** (interface de conversation) et le module d'**historique** (liste des sessions passées). La communication avec le backend s'effectue via le protocole **HTTP** avec transmission du token JWT pour l'authentification.

B) Backend Flask

Le backend est une **API REST** développée avec **Flask** (Python) et déployée sur Render, et qui gère les **endpoints** d'authentification, de gestion des conversations et de traitement des messages. Ce système repose essentiellement sur le pipeline **RAG** : à chaque requête de l'utilisateur, le backend effectue une **recherche sémantique** dans l'index FAISS pour identifier les passages les plus pertinents, construit un prompt enrichi avec le **contexte** récupéré et l'historique de la conversation, puis appelle le LLM via son API. Enfin, la réponse est générée et affichée via l'interface graphique de l'application.

C) Couche données

En ce qui concerne le **stockage des données**, nous avons opté pour une séparation en deux bases PostgreSQL, soit avec une base applicative ainsi qu'une base pour les données documentaires.

Ainsi, la **base applicative** contient les données des utilisateurs, de leurs messages et de leurs conversations. Elle gère de cette façon l'ensemble du cycle de vie des utilisateurs et de leurs échanges avec le chatbot.

La **base documentaire** quant à elle contient les différents fichiers PDF et les extraits (chunks) qui stockent les métadonnées des PDF indexés ainsi que le lien vers l'index FAISS. L'index vectoriel FAISS est stocké en local et chargé en mémoire au démarrage du serveur Flask.

V. Intégration du LLM et RAG

A) Configuration des modèles

Notre chatbot intègre un système LLM basé sur **OpenRouter**, qui est une plateforme d'agrégation de modèles de langage. Cette approche permet une stratégie de fallback **robuste** et offre une **flexibilité** et une **résilience**, en permettant le basculement automatique entre plusieurs modèles si un modèle n'est pas disponible.

Ainsi, le système supporte trois modèles LLM organisés dans l'ordre suivant :

- DeepSeek V4 Flash : Modèle optimisé pour les réponses rapides
- Llama 3.3 70B : Modèle haute capacité de Meta
- OpenRouter Free : Fallback gratuit

B) Système RAG

a) Principe général

L'architecture **RAG** (Retrieval-Augmented Generation) repose sur le principe d'enrichir chaque requête avec des **extraits pertinents** issus d'une base documentaire de référence. Le LLM joue alors le rôle d'un synthétiseur en recevant à la fois la question de l'utilisateur et le contexte documentaire récupéré, et produit une réponse à partir de ces différentes sources.

b) Ingestion des données

L'ingestion consiste à transformer les fichiers PDF bruts en une **structure vectorielle**, et se déroule en quatre étapes successives. Tout d'abord il y a la découverte des sources, où le script parcourt récursivement le répertoire configuré via la **variable d'environnement** et identifie l'ensemble des fichiers PDF présents. Seuls les fichiers PDF sont traités, ce qui évite d'indexer des fichiers non pertinents.

Ensuite il y a le découpage des fichiers en **chunks**, où le texte extrait est découpé en fragments de 512 tokens avec un chevauchement de 50 tokens entre fragments consécutifs. Ce chevauchement est essentiel pour éviter de couper une instruction ou une liste d'ingrédients en plein milieu d'un chunk, ce qui dégraderait la cohérence du contexte fourni au LLM.

Puis il y a la **génération des embeddings**, où chaque chunk est transformé en vecteur, et enfin le **stockage persistant**. En effet, l'index FAISS est sauvegardé sous la forme de deux fichiers complémentaires, soit un fichier .faiss qui contient les vecteurs et un fichier .pkl qui stocke les métadonnées associées (contenu textuel, source, numéro de page). Cela nous permet de charger l'index en mémoire au démarrage du serveur Flask, sans avoir à réingérer les documents à chaque redéploiement.

c) Pipeline de réponse en ligne

À chaque message envoyé par l'utilisateur, le pipeline de réponse s'exécute de façon séquentielle en cinq étapes. Tout d'abord, la question de l'utilisateur est transformée en vecteur par le **modèle d'embedding**, en utilisant strictement le même modèle que celui employé lors de l'ingestion.

En effet, une incompatibilité entre les deux modèles rendrait la recherche vectorielle incohérente. Le vecteur obtenu est ensuite comparé à l'ensemble de l'index FAISS par recherche des chunks les plus proches **sémantiquement**. Les chunks récupérés sont ensuite compilés en un bloc de contexte structuré, incluant pour chaque fragment son contenu textuel, le fichier source et le numéro de page d'origine. Ce formatage permet ainsi au LLM de **distinguer** clairement les différentes sources et facilite donc la génération de réponses pertinentes.

Un prompt complet est alors assemblé en quatre parties : un message système définissant le rôle de l'assistant culinaire et lui demandant de se limiter au **contexte fourni**, l'historique récent de la conversation, le contexte documentaire récupéré, et enfin la question de l'utilisateur. Ce prompt est envoyé au LLM sélectionné via l'API OpenRouter, pour une restitution de la réponse vers le frontend.

Ainsi, la réponse retournée à l'utilisateur est enrichie. Elle comprend le texte généré par le LLM, le contexte formaté avec numéros de références, et les métadonnées complètes des documents utilisés.

d) Avantages de l'architecture

Cette architecture présente plusieurs avantages essentiels pour un chatbot spécialisé dans le domaine de la cuisine. En effet, les réponses sont directement ancrées dans le corpus de recettes fourni, ce qui garantit leur **pertinence** et leur **exactitude** dans le domaine. Par ailleurs, la résilience est assurée par la possibilité de **basculer** entre DeepSeek V4 et LLaMA selon la disponibilité des modèles sur OpenRouter.

De plus, l'historique conversationnel intégré au prompt permet au chatbot de **comprendre** les questions sans que l'utilisateur ait à reformuler le contexte. Enfin, la recherche par similarité sémantique plutôt que par mots-clés permet de retrouver des passages pertinents même lorsque la question est formulée différemment du texte des recettes.

Ainsi, ce système offre une architecture RAG complète et robuste pour fournir des **réponses fiables et contextualisées** à partir de la documentation PDFs disponibles.

VI. Conteneurisation et déploiement

A) Conteneurisation

a) Stratégie de conteneurisation

Pour la **conteneurisation** de l'application, nous avons opté pour Docker, qui permet d'**encapsuler** chaque composant du système dans un environnement isolé et reproductible. Ainsi, l'objectif est de garantir que l'application se comporte de façon identique en **développement local et en production**. Dans le cadre de ce projet, la conteneurisation concerne principalement le backend Flask, qui concentre la logique métier, le pipeline RAG et les dépendances Python les plus complexes.

b) DockerFile

Chaque service utilise un build multi-stage pour **optimiser** la taille et la sécurité des images finales. Ainsi, le Dockerfile du backend est composé tout d'abord d'un **Builder**, qui installe les outils de compilation nécessaires aux wheels compilées comme FAISS, crée un environnement virtuel Python et installe toutes les dépendances et Gunicorn (serveur WSGI production).

De plus, le Dockerfile est également composé d'un **Runtime**, qui copie seulement l'environnement virtuel compilé depuis le builder, qui installe les dépendances binaires minimalistes et prépare les répertoires inscriptibles pour le RAG et les embeddings.

Concernant le Frontend, le Dockerfile dispose aussi d'un Builder, qui installe les **dépendances** npm et compile l'application Angular en production, ainsi que d'un Runtime, qui copie seulement le répertoire compilé, ne définit pas de `node_modules` dans l'image (économie de mémoire) et configure les **variables d'environnement** (port, URL du backend).

c) Fichier .dockerignore

Nous avons également ajouté un fichier .dockerignore, afin d'exclure de l'image les fichiers inutiles en production, dans l'optique d'**accélérer** la construction de l'image et de **réduire sa taille**.

Ainsi, certains répertoires sont exclus car les PDFs sources et l'index FAISS ne sont pas embarqués dans l'image Docker. En effet, ils sont respectivement stockés en externe et rechargés **dynamiquement** au démarrage du serveur.

B) Déploiement de l'application

Nous avons opté pour Render, qui est une plateforme retenue d'hébergement d'applications. Ce choix a notamment été fait car elle supporte nativement le déploiement depuis une image Docker, ce qui nous garantissait une **cohérence** entre l'environnement de développement et l'environnement de production. Ainsi, nous avons configuré le déploiement à l'aide d'un fichier render.yaml, versionné dans le dépôt, et qui décrit de façon déclarative les services à déployer.

Néanmoins, sur le free tier de Render, le service se met en veille après 15 minutes d'inactivité, entraînant un délai de démarrage d'environ 30 secondes à la première requête suivante. Afin de limiter cet effet, nous avons configuré un service de monitoring externe pour envoyer une requête **HTTP** sur l'endpoint de health check toutes les 10 minutes, maintenant ainsi le serveur actif en permanence.

VIII. Défis et difficultés majeures

A) Qualité des réponses RAG

La principale difficulté technique du projet a concerné la **qualité des réponses** générées par le pipeline RAG. En effet, lors des premières phases de test, le chatbot produisait fréquemment des réponses **imprécises**, hors sujet ou incomplètes, surtout quand la question de l'utilisateur était formulée différemment du vocabulaire employé dans les recettes en PDF. Dans certains cas, le modèle générait des réponses plausibles mais pas ancrées dans le corpus documentaire, ce que l'on appelle des **hallucinations**.

Suite à une analyse approfondie du problème, nous avons pu identifier plusieurs causes. Tout d'abord, la **taille initiale des chunks** était trop grande, ce qui diluait l'information pertinente dans des passages trop longs et rendait difficile la récupération d'une information précise. De plus, le modèle d'embedding utilisé initialement n'était pas le plus adapté pour le français, produisant des représentations vectorielles de mauvaise qualité.

Par ailleurs, le LLM ne récupérait pas toujours suffisamment de contexte pour les questions complexes impliquant notamment plusieurs étapes d'une recette. Enfin, il suffisait que les prompts ne correspondent pas exactement aux documents ou au contexte pour que le LLM provoque des hallucinations.

B) Gestion du CORS

Ensuite, la mise en place de la **communication** entre le frontend Angular et le backend Flask a posé des difficultés, notamment liées au mécanisme de sécurité **CORS** (Cross-Origin Resource Sharing). En effet, les navigateurs web bloquent par défaut toute requête HTTP vers un domaine différent de celui de la page courante, ce qui empêchait notamment Angular d'appeler l'API Flask en production. Ainsi, les erreurs se manifestaient de façon silencieuse côté utilisateur, avec des requêtes bloquées sans message d'erreur explicite dans l'interface, rendant la détection et le diagnostic difficiles.

La difficulté était d'autant plus complexe à régler que les appels fonctionnaient parfaitement en local, avec frontend et backend qui tournent en local avec des ports différents. Le problème n'est donc apparu qu'au moment du premier déploiement en production, soit à un stade avancé du projet. Plus précisément, les requêtes HTTP classiques (POST, GET, DELETE) posaient notamment problème, puisqu'elles étaient bloquées, faute d'en-têtes CORS appropriés dans les réponses Flask.

VIII. Conclusion

Nous avons ainsi pu à l'issue de ce projet, **concevoir, développer et déployer** une application web complète de chatbot intelligent spécialisé dans le domaine de la cuisine. En effet, à partir d'un cahier des charges impliquant une **interface conversationnelle**, une **authentification** utilisateur et une base de connaissances documentaire, nous avons construit une solution fonctionnelle reposant sur une architecture RAG (Retrieval-Augmented Generation), combinant recherche vectorielle et génération par LLM.

Par ailleurs, nous sommes parvenus à implémenter et déployer l'ensemble des fonctionnalités définies lors de l'**analyse des besoins** au début du cycle du projet. Ainsi, l'utilisateur peut créer un compte, s'authentifier, poser des questions en français sur des recettes de cuisine et recevoir des réponses contextualisées en temps réel, à partir du corpus documentaire indexé.

De plus, l'**historique des conversations** est sauvegardé et accessible à tout moment, sans oublier le pipeline RAG, qui ingère les fichiers PDF, découpe leur contenu en fragments (chunks), génère leurs représentations vectorielles et les indexe dans FAISS pour permettre une recherche sémantique efficace à chaque requête.

Mais au-delà du livrable technique, ce projet a été une opportunité d'**apprentissage** dense et transversale pour l'ensemble de l'équipe. En effet, nous avons pu apprendre et mettre en pratique un ensemble de technologies modernes et complémentaires rarement abordées jusqu'alors, comme le RAG, mais également le déploiement conteneurisé avec Docker.

Enfin, sur le plan organisationnel, le travail en équipe autour d'outils professionnels comme **Jira** et **Git** nous a permis de travailler dans des conditions similaires aux **projets d'entreprise**. En effet, la pratique des **pull requests** et des **code reviews** a favorisé le partage de connaissances, ce qui est essentiel aussi bien dans le cadre académique que dans le monde professionnel.